



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. március 5.

- 1 Oszd meg és uralkodj
 - Oszd meg és uralkodj algoritmusok
 - Összefésülő rendezés

- 2 Összefésülő rendezés
 - Felosztás lépése
 - Tömbös ábrázolás
 - Uralkodás és összefésülés
 - Összegzés

Oszd meg és uralkodj

Oszd meg és uralkodj algoritmusok

- *Az oszd meg és uralkodj algoritmus egy nagy problémát kisebb részproblémákra bont, amelyek maguk is tovább bonthatók még kisebb részproblémákra, és így tovább. A rekurzió akkor ér véget, amikor eléri az alapesetet, és a részprobléma elég kicsi ahhoz, hogy közvetlenül megoldható legyen további rekurzió nélkül.*
- Az oszd meg és uralkodj algoritmus lépései:
 - **Felosztás:** A problémát egy vagy több kisebb, azonos típusú részproblémára bontjuk.
 - **Uralkodás:** A részproblémákat rekurzívan megoldjuk.
 - **Összefésülés:** A részproblémák megoldásait kombinálva kapjuk meg az eredeti probléma megoldását.

- Fő ötlet:

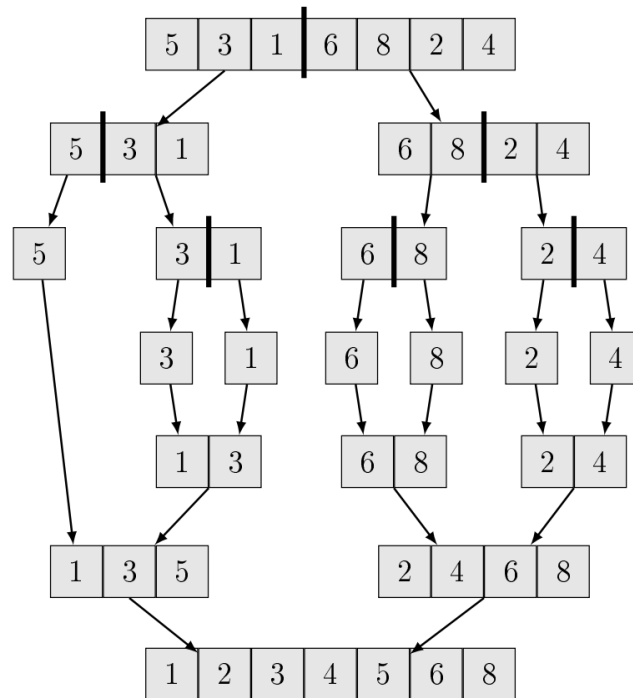
Képzeljük el, hogy két már rendezett tömbünk van. Van-e hatékony módja annak, hogy ezekből egy új, rendezett tömböt hozzunk létre?

- Összefésülő rendezés:

Az összefésülési rendezés egy rekurzív algoritmus, amely folyamatosan két részre osztja a tömböt, amíg az tovább nem bontható, azaz csak egyetlen elemből áll (egy egyelemű tömb mindig rendezett). Ezután a rendezett rész-tömböket egyetlen rendezett tömbbé egyesíti.

Összefésülő rendezés

Összefésülő rendezés - Ábrázolás



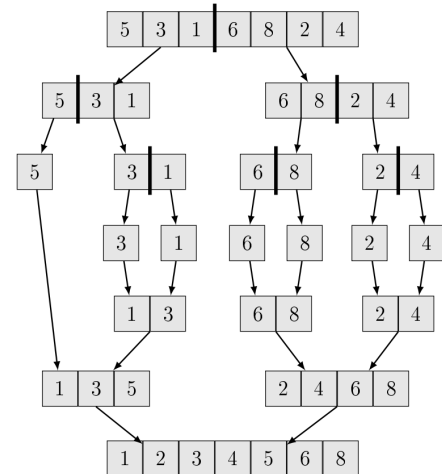
Összefésülő rendezés - Felosztás lépése

$\text{mergeSort}(A : \mathcal{T}[n])$

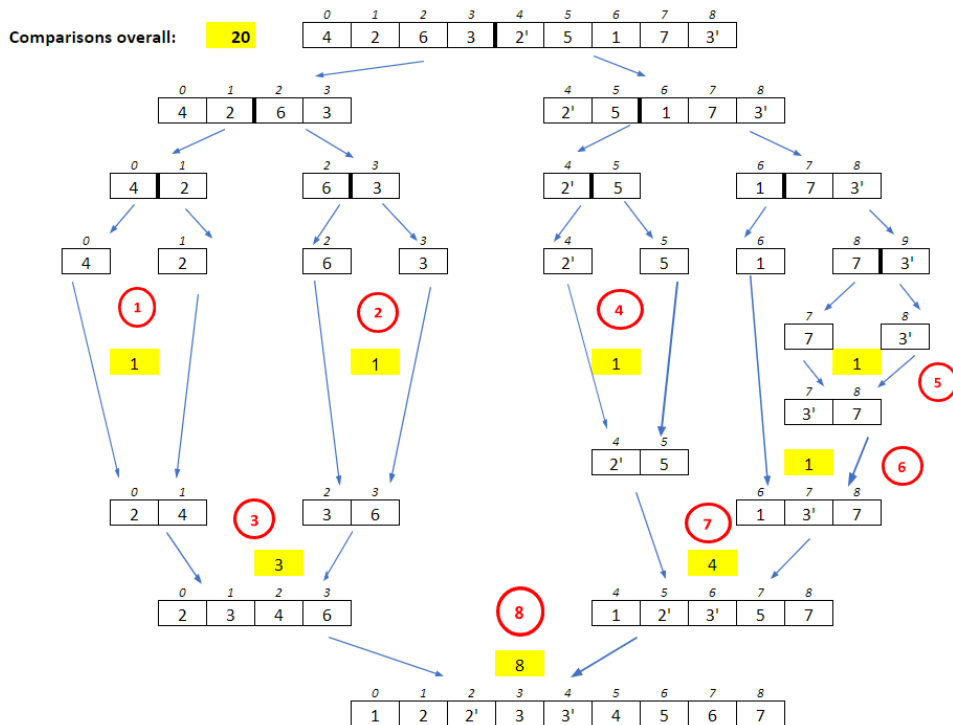
$B : \mathcal{T}[n] ; B[0..n] := A[0..n]$
// Sort $B[0..n]$ into $A[0..n]$ non-decreasingly:
$\text{ms}(B, A, 0, n)$

$\text{ms}(B, A : \mathcal{T}[] ; u, v : \mathbb{N})$

// Initially $B[u..v] = A[u..v]$.	
// Sort $B[u..v]$ into $A[u..v]$ non-decreasingly:	
$v - u > 1$	
$m := \lfloor \frac{u+v}{2} \rfloor$	SKIP
$\text{ms}(A, B, u, m)$ // Sort $A[u..m]$ into $B[u..m]$ non-decreasingly.	
$\text{ms}(A, B, m, v)$ // Sort $A[m..v]$ into $B[m..v]$ non-decreasingly.	
$\text{merge}(B, A, u, m, v)$ // merge $B[u..m]$ and $B[m..v]$ into $A[u..v]$	



Összefésülő rendezés - Tömbös ábrázolás



Összefésülő rendezés - Tömbös ábrázolás

1

	0	1	2	3	4	5	6	7	8
A	4	2	6	3	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

2

	0	1	2	3	4	5	6	7	8
A	2	4	6	3	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

3

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

4

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

	0	1	2	3	4	5	6	7	8	
merge(B,A,0,1,2)	2	4	6	3	2'	5	1	7	3'	1
(from B to A)	4	2	6	3	2'	5	1	7	3'	

	0	1	2	3	4	5	6	7	8	
merge(B,A,2,3,4)	2	4	3	6	2'	5	1	7	3'	1
(from B to A)	4	2	6	3	2'	5	1	7	3'	

	0	1	2	3	4	5	6	7	8	
merge(A,B,0,2,4)	2	4	3	6	2'	5	1	7	3'	3
(from A to B)	2	3	4	6	2'	5	1	7	3'	

	0	1	2	3	4	5	6	7	8	
merge(B,A,4,5,6)	2	4	3	6	2'	5	1	7	3'	1
(from B to A)	2	3	4	6	2'	5	1	7	3'	

Összefésülő rendezés - Tömbös ábrázolás

5

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

merge(A,B,7,8,9)
(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

1

START

6

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

merge(B,A,6,7,9)
(from B to A)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

1

START

7

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

merge(A,B,4,6,9)
(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

4

START

8

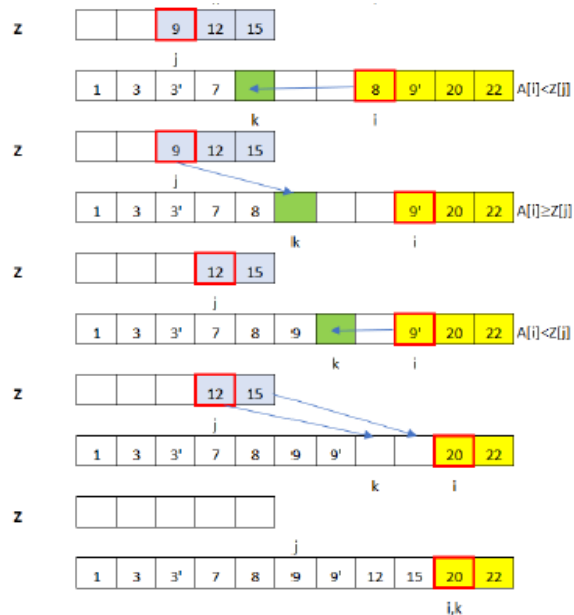
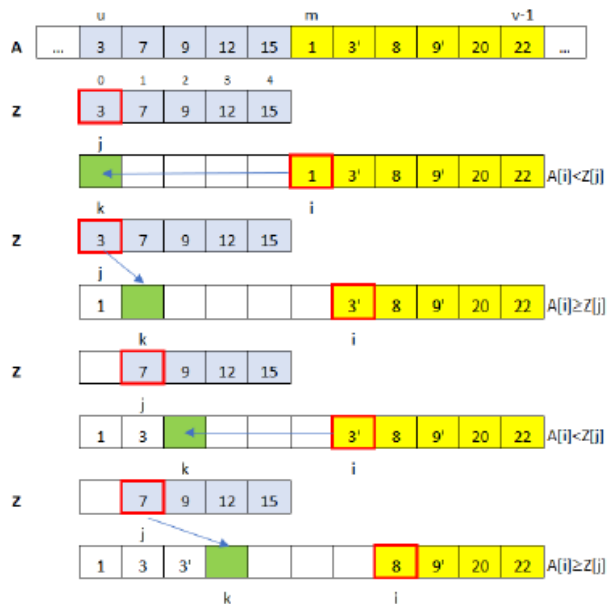
	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

merge(B,A,0,4,9)
(from B to A)

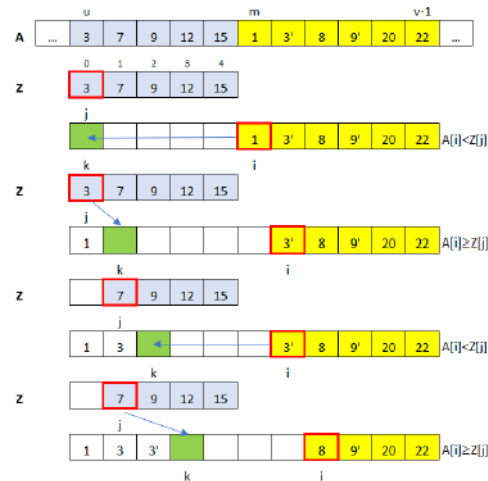
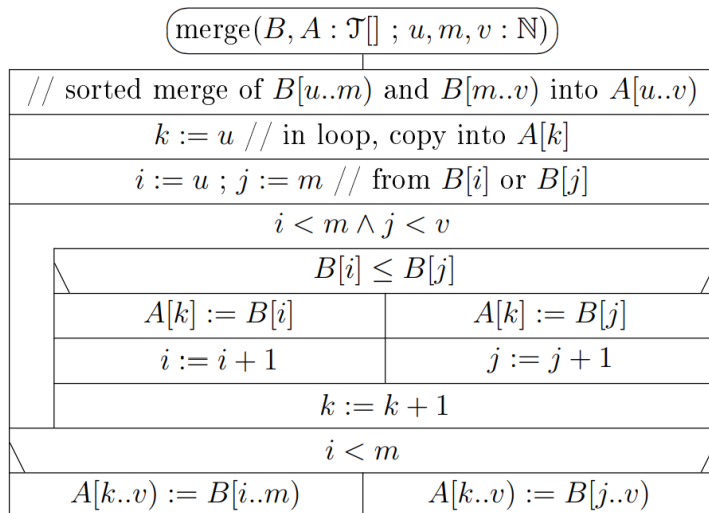
	0	1	2	3	4	5	6	7	8
A	1	2	2'	3	3'	4	5	6	7
B	2	3	4	6	1	2'	3'	5	7

8

Összefésülő rendezés - Helytakarékoság



Összefésülő rendezés - Uralkodás és összefésülés



Összefésülő rendezés - Összegzés

Műveletigénye:

- $mT(n), AT(n), MT(n) \in O(n * \log(n))$

Tárhely igénye:

- $O(n)$
- Szükség van egy ideiglenes tömb létrehozására a rendezett tömbök összefésüléséhez.

Stabilitás: A rendezési algoritmus megőrzi az azonos értékű elemek eredeti sorrendjét.

Köszönöm a figyelmet!